PHP : ALLER PLUS LOIN

Erreurs & exceptions · Namespace · Trait

Application support : Gestion des Véhicules V2

Lab de découverte — Solicode

Objectifs du lab

À la fin de cette présentation, je peux :

- **Détecter et traiter** une erreur sans interrompre l'application brutalement.
- **Organiser** les classes PHP avec des namespaces.
- **Réutiliser** un comportement commun avec un trait.
- Appliquer ces notions au backend PHP de **Gestion des Véhicules V2**.

Contexte technique : API PHP, opérations CRUD et données de véhicules en JSON.

01 — Erreurs et exceptions

Pourquoi les gérer ?

Une API peut recevoir un JSON invalide, ne pas trouver un véhicule ou rencontrer
un problème de lecture de fichier.

- Donner une réponse compréhensible au client.
- Choisir un code HTTP adapté.
- Enregistrer les détails utiles au diagnostic.
- Éviter d'afficher des détails internes à l'utilisateur.

Deux notions à distinguer

- **Erreur PHP :** problème signalé par le moteur, par exemple une erreur de type.
- **Exception :** objet lancé volontairement avec `throw`, puis traité avec `catch`.
- `Throwable` est l'interface commune aux erreurs PHP et aux exceptions.

Important : on ne doit pas essayer de continuer après toutes les erreurs.

Exemple — JSON de véhicule invalide

`JSON_THROW_ON_ERROR` transforme l'échec du décodage en exception `JsonException`.

```php
<?php

try {
    $vehicle = json_decode(
        $requestBody,
        true,
        512,
        JSON_THROW_ON_ERROR
    );

    if (!is_array($vehicle) || empty($vehicle['registration'])) {
        http_response_code(422);
        echo json_encode(['error' => 'Immatriculation obligatoire']);
    } else {
        // Continuer la création du véhicule.
    }
} catch (JsonException $exception) {
    error_log($exception->getMessage());
    http_response_code(400);
    echo json_encode(['error' => 'Le JSON envoyé est invalide']);
}
```

Résultat attendu : JSON invalide → réponse `400` ; champ métier manquant → `422`.

Exemple — véhicule introuvable

On peut créer une exception dédiée au domaine du projet :

```php
<?php
namespace App\Exceptions;

use RuntimeException;

class VehicleNotFoundException extends RuntimeException
{
}
```

Le contrôleur la traite précisément :

```php
try {
    $vehicle = $vehicleRepository->findOrFail($id);
} catch (VehicleNotFoundException $exception) {
    http_response_code(404);
    echo json_encode(['error' => 'Véhicule introuvable']);
}
```

À tester : demander un identifiant existant, puis un identifiant absent.

02 — Namespace

Un namespace est un **espace de noms** qui organise les classes et évite les collisions.

```php
<?php
namespace App\Models;

class Vehicle
{
    public function __construct(
        public string $registration,
        public string $brand
    ) {
    }
}
```

Dans un contrôleur, on importe la classe :

```php
<?php
namespace App\Http\Controllers;

use App\Models\Vehicle;
use App\Repositories\VehicleRepository;
```

`App\Models\Vehicle` est le nom complet de la classe.

Namespace dans le projet

Les dossiers peuvent refléter les namespaces :

```
app/
├── Exceptions/VehicleNotFoundException.php
├── Http/Controllers/VehicleController.php
├── Models/Vehicle.php
└── Repositories/VehicleRepository.php
```

Avec Composer, l'autoload PSR-4 associe le préfixe `App\` au dossier `app/` :

```
{
  "autoload": {
    "psr-4": {
      "App\\": "app/"
    }
  }
}
```

Après une modification de l'autoload : `composer dump-autoload`

03 — Trait

Un **trait** partage des méthodes entre plusieurs classes, sans créer de relation d'héritage.

Exemple : les contrôleurs Véhicule et Catégorie renvoient tous deux du JSON.

```php
<?php
namespace App\Http\Support;

trait JsonResponse
{
    protected function respondWithJson(array $data, int $status = 200): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($data, JSON_THROW_ON_ERROR);
    }
}
```

Le trait centralise la préparation des réponses API JSON.

Utiliser le trait dans les contrôleurs

```php
<?php
namespace App\Http\Controllers;

use App\Http\Support\JsonResponse;

class VehicleController
{
    use JsonResponse;

    public function show(array $vehicle): void
    {
        $this->respondWithJson(['data' => $vehicle]);
    }
}
```

Un autre contrôleur peut aussi écrire `use JsonResponse;`.

- **Avantage :** éviter de copier-coller le même comportement.
- **Limite :** un trait ne doit pas devenir un endroit où ranger toute la logique.
- **À vérifier :** si l'encodage échoue, `JsonException` doit être traité par le gestionnaire d'erreurs de l'application.

Mise en pratique — scénario CRUD

Étape	Notion utilisée	Résultat attendu
Créer un véhicule avec un JSON mal formé	Exception	Réponse `400` et erreur journalisée
Créer un véhicule sans immatriculation	Validation	Réponse `422`
Afficher un identifiant inexistant	Exception métier	Réponse `404`
Charger `Vehicle` dans un contrôleur	Namespace	Classe trouvée via `App\Models`
Répondre depuis plusieurs contrôleurs	Trait	Même format JSON réutilisé

Test manuel : envoyer les cas valides et invalides avec un client HTTP (par exemple Postman), puis vérifier le statut et le corps JSON.

À retenir

- **Erreurs et exceptions** : détecter un problème, le traiter au bon niveau, journaliser ce qui aide au diagnostic et répondre clairement.
- **Namespace** : nommer et organiser les classes pour éviter les conflits.
- **Trait** : partager une méthode entre classes sans dupliquer le code.

La bonne règle : chaque outil doit répondre à un besoin réel du projet.

MERCI !

Questions ?

PHP · API · Gestion des Véhicules V2